Course Name: (PCCCS495) Advanced Programming (OOP) Lab

Project Title: Student Attendance Management System

Student Name: Avinav Burman

Roll Number: 76

Instructor Name: Soumadip Biswas

Semester: Spring 2026

2. Problem Statement

Manual attendance tracking systems are inefficient, error-prone, and lack transparency. Maintaining attendance using paper or spreadsheets can lead to data duplication, incorrect calculations, and difficulty in generating reports.

The objective of this project is to design and implement a **Student Attendance Management System** that automates attendance tracking using Object-Oriented Programming principles.

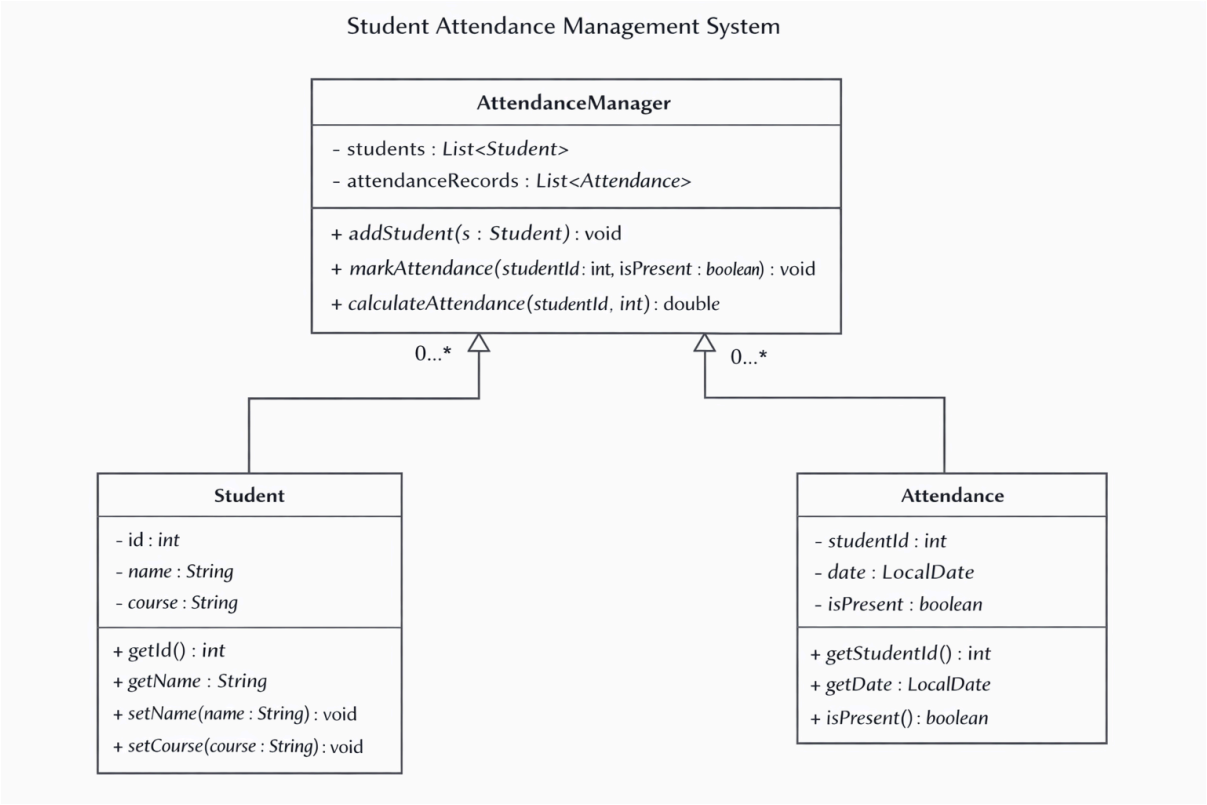
Key Objectives:

- Provide CRUD operations for student management
- Enable daily attendance marking (Present/Absent)
- Automatically calculate attendance percentage
- Store data persistently using file handling
- Generate attendance reports
- Ensure robust validation and exception handling

This system improves efficiency, ensures accuracy, and provides a structured approach to attendance management.

3. System Design

3.1 UML Class Diagram



3.2 Description of Major Classes

Student Class

- Stores student details: id, name, course
- Validates input in constructor and setters
- Provides controlled access using getters

Attendance Class

- Represents a single attendance record
- Stores studentId, date, and attendance status
- Designed as an immutable data object

AttendanceManager Class

- Core service layer handling business logic
- Manages student and attendance lists
- Performs CRUD operations and calculations
- Handles file storage and retrieval
- Generates attendance reports

Main Class

- Provides menu-driven CLI interface
- Handles user input using Scanner
- Calls service methods and handles exceptions

Exception Classes

- DuplicateStudentException
- InvalidDataException
- StudentNotFoundException

3.3 Package Structure

- model → Student, Attendance
- service → AttendanceManager
- exception → Custom exceptions
- main → Application entry point

3.4 Interaction Flow

1. Application starts
2. Data loaded from files (students.txt, attendance.txt)
3. User selects operation from menu
4. Service layer processes request
5. Data updated and saved
6. Output displayed

4. OOP Concepts Demonstrated

4.1 Encapsulation

Where Used: Student & Attendance classes

```
private String name;
public void setName(String name) {
        if (name == null || name.trim().isEmpty()) {
        throw new IllegalArgumentException("Invalid name");
        }
        this.name = name;
}
```

Explanation:
Data members are private and accessed via methods, ensuring validation and data integrity.

4.2 Abstraction

Where Used: AttendanceManager

```
double percent = manager.calculateAttendance(id);
```

Explanation:
The internal calculation logic is hidden from the user; only the result is exposed.

4.3 Separation of Concerns

- Model → Data
- Service → Logic
- Main → UI

Explanation:
Improves maintainability and modularity.

4.4 Custom Exceptions (Polymorphism)

```
throw new StudentNotFoundException("Student not found");
```

Explanation:
Different exceptions represent different error scenarios, improving clarity and debugging.

5. Implementation Highlights

5.1 Add Student with Validation

```
public void addStudent(Student s) throws DuplicateStudentException {
        for (Student student : students) {
        if (student.getId() == s.getId()) {
        throw new DuplicateStudentException("Duplicate ID");
        }
        }
        students.add(s);
}
```

Justification: Prevents duplicate entries and ensures valid data.

5.2 Attendance Marking

```
attendanceRecords.add(
        new Attendance(studentId, LocalDate.now(), isPresent)
);
```

Justification: Records real-time attendance efficiently.

5.3 Attendance Calculation

```
return total == 0 ? 0 : (present * 100.0 / total);
```

Justification: Prevents division-by-zero errors.

5.4 File Handling (Students)

```
writer.println(s.getId() + "," + s.getName() + "," + s.getCourse());
```

Justification: Uses CSV format for lightweight storage.

5.5 Report Generation

```
writer.printf("Attendance: %.2f%% (%d/%d)\n", percentage, present, total);
```

Justification: Provides structured and readable reports.

6. Testing & Error Handling

1. **Test Cases**
    - Add student successfully
    - Duplicate student → exception
    - Mark attendance
    - Calculate percentage

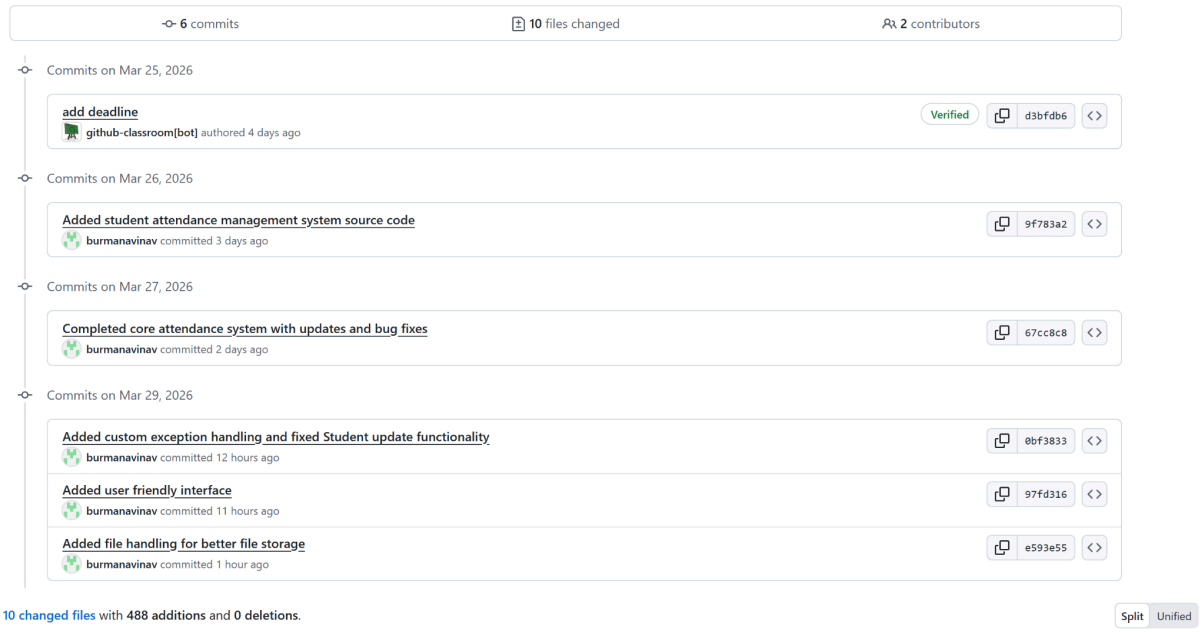
2. **Edge Cases**
    - No attendance records → returns 0%
    - Missing files → handled gracefully
    - Corrupted file lines → skipped

3. **Failure Scenarios**
    - Duplicate ID → DuplicateStudentException
    - Invalid input → InvalidDataException
    - Student not found → StudentNotFoundException

4. **Identified Limitation**
    - Multiple attendance entries allowed per day per student

7. Git Workflow

Commit History Screenshot



Explanation of Development Progression

The project was developed incrementally using Git, with each major feature implemented and committed separately.

1. Initial Setup

- Repository initialized using GitHub Classroom
- Basic project structure created

Commit: *"add deadline"* (auto-generated)

---

2. Initial Code Implementation

Added core source code for:

- ❖ Student
- ❖ Attendance
- ❖ AttendanceManager

Commit:
"Added student attendance management system source code"

---

3. Core System Development

- Implemented main features:
  - Add student
  - Mark attendance
  - Calculate attendance
- Fixed initial bugs

Commit:
"Completed core attendance system with updates and bug fixes"

---

4. Exception Handling & Update Feature

- Added custom exceptions:
  - DuplicateStudentException
  - InvalidDataException
  - StudentNotFoundException
- Fixed student update functionality

Commit:
"Added custom exception handling and fixed Student update functionality"

---

5. User Interface Implementation

- Developed menu-driven CLI interface
- Improved user interaction

Commit:
"Added user friendly interface"

---

6. File Persistence

- Implemented file handling:
  - students.txt
  - attendance.txt
- Enabled data persistence

Commit:
"Added file handling for better file storage"]

Git was used to track the development of the project in a structured manner. Each feature was implemented incrementally and committed separately, ensuring better version control, easier debugging, and clear project progression

8. Conclusion

The system successfully automates attendance tracking using OOP principles, file handling, and exception management. It reduces manual errors and improves efficiency.

Future Scope

- GUI or Web interface
- Database integration (MySQL/SQLite)
- Multi-user support
- QR/Biometric attendance
- Prevent duplicate daily attendance
- Unit testing using JUnit

---

9. Appendix

PCCCS495 Proposal Review – APPROVED_MINOR

S

soumadip.biswas@ieminternational.ai

To: AvinavDeep Burman

Wed 3/11/2026 2:03 PM

Dear Avinav Deep Burman,

Project: Student Attendance Management System
Decision: APPROVED_MINOR
Score: 88

Strengths:
The proposal clearly identifies a prevalent problem and offers a practical, relevant solution. The features are well-aligned with the problem statement, and the target audience is clearly defined. The proposed MVC architecture with a DAO layer demonstrates a solid understanding of software design principles, promoting modularity, maintainability, and scalability. The inclusion of core OOP concepts like Abstraction, Inheritance, and Polymorphism is appropriate for a university Java OOP project. The system aims to significantly improve efficiency and transparency in attendance management.

Improvements:
While 'Automated' attendance marking is listed, specifying the underlying technology or method (e.g., QR code scanning, biometric integration, network-based detection) would provide greater clarity and strengthen this feature's feasibility. The mention of 'Threads' as a core OOP concept could be further justified by outlining a specific, critical use case beyond general UI responsiveness or background processing (e.g., concurrent data processing for real-time analytics, or managing multiple concurrent attendance inputs), to demonstrate a deeper application of concurrency in the system's core logic. Additionally, briefly mentioning potential data storage mechanisms (e.g., SQL database, flat files) in conjunction with the DAO layer could provide a more complete picture of the persistence strategy.

Regards,
Course Instructor